

Product Requirements Document (PRD)

Project Name

git-smart

Tagline:

AI-powered Git workflow assistant that generates production-quality commit messages, summarizes code changes, and streamlines Git operations.

Vision

Developers spend time writing commit messages, reviewing diffs, and managing repetitive Git workflows.

git-smart should become an intelligent CLI companion that:

- Understands repository state
- Performs Git pre-checks
- Reviews staged code changes
- Generates high-quality commit messages
- Summarizes modifications
- Supports Conventional Commits
- Commits changes
- Optionally pushes to remote

The tool should feel like a senior engineer reviewing the code before committing.

Primary Users

1. Solo developers
 2. Open source contributors
 3. Backend engineers
 4. Frontend engineers
 5. Students learning Git
 6. AI-assisted developers
-

Package Name

git-smart

NPM Package:

git-smart

CLI Command:

git-smart

Tech Stack

Runtime

Node.js >= 20

Reason:

- Excellent CLI ecosystem
 - Native ESM support
 - Cross-platform
-

Language

TypeScript

Reason:

- Type safety
 - Better maintainability
 - Easier contributions
-

CLI Framework

Commander.js

Responsibilities:

- Commands
 - Flags
 - Help menus
-

Interactive UI

Inquirer.js

Responsibilities:

- Menus
 - Confirmations
 - User inputs
-

Terminal Styling

Chalk

Responsibilities:

- Colors
 - Success messages
 - Errors
-

Spinners

Ora

Responsibilities:

- Loading indicators
-

Git Operations

simple-git

Responsibilities:

- Git status
- Branches
- Remotes
- Commits

Avoid shelling out wherever possible.

LLM Integration

Provider Agnostic

Supported:

- OpenAI
- OpenRouter
- Anthropic
- Gemini

Architecture:

Provider Adapter Pattern

```
src/providers/  
openai.ts  
openrouter.ts  
anthropic.ts  
gemini.ts
```

V1 Features

Feature 1

Repository Validation

Check

Is current directory a Git repository?

Implementation:

```
git rev-parse --git-dir
```

If False

Display:

"This directory is not a Git repository."

Options:

1. Initialize Repository
2. Exit

If Initialize

Execute:

git init

Feature 2

Git Ignore Validation

Check:

.gitignore exists?

If Missing:

Prompt:

"No .gitignore file found."

Generate one?

Options:

1. Node
2. Python
3. Go
4. PHP
5. Ruby
6. Rust
7. Java

8. Skip

Use official gitignore templates.

Store templates locally.

Feature 3

Remote Validation

Check:

`git remote -v`

Maintain:

`remoteExists=true|false`

If no remote:

Prompt:

"No remote repository configured."

Would you like to add one?

Options:

1. Yes
2. No

If Yes:

Prompt:

Remote URL:

Execute:

`git remote add origin URL`

Feature 4

Detect First Commit

Check:

```
git rev-parse --verify HEAD
```

If fails:

```
isFirstCommit=true
```

Feature 5

Repository State Analysis

Collect:

```
git status --short
```

Show:

Modified files

Deleted files

Added files

Untracked files

Example:

Modified:

```
src/app.ts
```

Added:

```
src/utils/logger.ts
```

Deleted:

```
src/old.ts
```

Feature 6

Stage Changes

Prompt:

Stage all changes?

Options:

1. Yes
 2. No
 3. Select files
-

If Select Files

Display checkbox list

Allow manual selection

Feature 7

Collect Diff

After staging:

`git diff --cached`

Never use:

`git diff`

Reason:

`git diff` becomes empty after staging.

Feature 8

Diff Statistics

Collect:

`git diff --cached --stat`

Collect:

`git diff --cached --numstat`

Metrics:

- files changed
 - insertions
 - deletions
-

Feature 9

File Classification

Classify files:

High Importance

controllers
routes
services
models
middlewares

Medium Importance

utils
configs

Low Importance

package-lock
yarn.lock
pnpm-lock

Ignore:

dist
build
coverage

Feature 10

AI Change Summary

Generate summary:

Example:

Changes Summary

✓ Added ApiError utility

✓ Added ApiResponse utility

✓ Added asyncHandler wrapper

✓ Refactored environment configuration

✓ Removed constants file

Feature 11

Commit Message Generation

Generate:

3 commit messages

Example:

1.

feat(core): add reusable API utility classes

2.

refactor(config): centralize environment setup

3.

chore(project): improve backend initialization flow

Feature 12

Conventional Commit Support

Prompt:

Use Conventional Commits?

Options:

1. Yes
2. No

If Yes:

Generate:

feat:

fix:

docs:

refactor:

test:

chore:

perf:

build:

ci:

Feature 13

Commit Message Editing

Prompt:

Select Commit Message

Options:

- 1
 - 2
 - 3
- Custom

If Custom

Allow editing

Feature 14

Commit

Execute:

```
git commit -m "message"
```

Feature 15

Push Workflow

If remoteExists=true

Prompt:

Push to remote?

Options:

1. Yes
 2. No
-

If Yes

Current branch:

```
git branch --show-current
```

Execute:

```
git push origin branch
```

CLI Flow

START

↓

Validate Repository

↓

Validate Git Ignore

↓

Validate Remote

↓

Detect First Commit

↓

Analyze Repository State

↓

Stage Files

↓

Collect Diff

↓

Generate Summary

↓

Generate Commit Messages

↓

User Selects Message

↓

Commit

↓

Push

↓

END

Commands

Default

git-smart

Full workflow

Dry Run

git-smart --dry-run

Generates summary and messages

No commit

No push

Summary Only

git-smart summary

Only generate change summary

Commit Only

git-smart commit

Generate commit messages

Commit

No push

Push

git-smart push

Push current branch

Configuration

Store:

.git-smart.json

Example:

```
{  
  "provider": "openrouter",  
  "model": "deepseek/deepseek-chat-v3",  
  "conventionalCommits": true,  
  "autoPush": false,  
  "showSummary": true  
}
```

Environment Variables

OPENAI_API_KEY

OPENROUTER_API_KEY

ANTHROPIC_API_KEY

GEMINI_API_KEY

AI Prompt

SYSTEM

You are a senior software engineer reviewing code before commit.

Analyze the provided git diff.

Requirements:

- Understand intent
- Ignore formatting-only changes
- Ignore lock file noise
- Focus on business logic
- Follow Conventional Commits when requested
- Generate concise production-grade commit messages

Return JSON only.

Schema:

```
{
  "summary": [
    "item1",
    "item2"
  ],
  "impact": "low|medium|high",
  "messages": [
    "message1",
    "message2",
    "message3"
  ]
}
```

USER TEMPLATE

Repository Context:

{{repo_state}}

Files Changed:

{{files}}

Diff Statistics:

{{stats}}

Git Diff:

{{diff}}

Generate:

1. Summary
2. Impact Score
3. Three commit messages

Output Example

Impact: Medium

Summary

- ✓ Added ApiError utility
- ✓ Added ApiResponse utility
- ✓ Added asyncHandler wrapper
- ✓ Refactored environment loader

Suggested Commits

1. feat(core): add reusable API utilities
2. refactor(config): centralize environment management
3. chore(project): improve backend bootstrap structure

Select:

Success Metrics

1. Commit message accepted without editing > 60%
 2. Workflow completion > 90%
 3. Runtime < 5 seconds excluding LLM
 4. Cross-platform support
 5. Zero Git corruption risk
-
-

Release Goal

Version 1.0.0 should be publishable as:

```
npm install -g git-smart
```

and immediately usable in production repositories.